

XGBoost (eXtreme Gradient Boosting)

Paweł Teisseyre

Faculty of Mathematics and Information Science



XGBoost Algorithm

- The algorithm has gained enormous popularity in recent years, winning many competitions.
- The authors of the algorithm are Tianqi Chen and Carlos Guestrin (University of Washington), description can be found in the paper **XGBoost: A Scalable Tree Boosting System**:
<https://arxiv.org/pdf/1603.02754.pdf>.
- Open source implementation: <https://github.com/dmlc/xgboost>
- XGBoost is available in: Python, R, Julia.
- Distinguishing features of XGBoost:
 - ① Ability to optimize any loss function
 - ② Approximation of the risk function based on second-order Taylor expansion.
 - ③ Regularization function, including tree size and weights assigned to individual leaves.
 - ④ New tree construction algorithm, new criterion for splitting observations in nodes.
 - ⑤ Many additional technical elements.

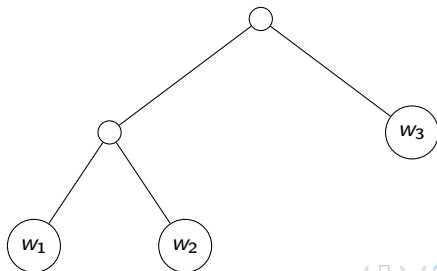
XGBoost Algorithm

- Prediction based on K functions:

$$\hat{y}_i = \phi(x_i) = \sum_{k=1}^K f_k(x_i), \quad f_k \in \mathcal{F}$$

where $\mathcal{F} = \{f(x) = w_{q(x)}\}$ is a family of trees, q represents the tree structure, while w is a vector of weights for the leaves.

- Tree structure: $q : R^p \rightarrow \{1, \dots, T\}$, $w \in R^T$, while T is the number of leaves.
- Example: $w = (w_1, w_2, w_3)$.



XGBoost Algorithm

- Prediction based on K functions:

$$\hat{y}_i = \phi(x_i) = \sum_{k=1}^K f_k(x_i), \quad f_k \in \mathcal{F}$$

where $\mathcal{F} = \{f(x) = w_{q(x)}\}$ is a family of trees, q represents the tree structure, while w is a vector of weights for the leaves.

- Tree structure: $q : R^p \rightarrow \{1, \dots, T\}$, $w \in R^T$, while T is the number of leaves.

We consider the problem of **empirical risk minimization**:

$$L(\phi) = \sum_{i=1}^n \underbrace{l(\hat{y}_i, y_i)}_{\text{loss function}} + \sum_k \underbrace{\Omega(f_k)}_{\text{regularization}},$$

where

$$\Omega(f) = \gamma T + \frac{1}{2} \lambda \|w\|^2$$

and γ, λ are parameters.

XGBoost Algorithm

- Our goal is to construct a sequence of models $f_1(x), f_2(x), \dots, f_K(x)$.
- Let $\hat{y}_i^{(t)}$ denote the prediction at iteration t .
- At iteration t we add f_t to minimize the **regularized empirical risk function**:

$$\mathcal{L}^{(t)} = \sum_{i=1}^n \underbrace{l(y_i, \hat{y}_i^{(t-1)} + f_t(x_i))}_{\text{loss after adding } f_t \text{ to the ensemble}} + \Omega(f_t)$$

- This means that we greedily add f_t that most reduces the value of the empirical risk.
- The above optimization problem with respect to f_t is difficult to solve, we need some additional approximations.

XGBoost Algorithm

- At iteration t we add f_t to minimize the **regularized empirical risk function**:

$$\mathcal{L}^{(t)} = \sum_{i=1}^n \underbrace{l(y_i, \hat{y}_i^{(t-1)} + f_t(x_i))}_{\text{loss after adding } f_t \text{ to the ensemble}} + \Omega(f_t)$$

- We apply a second-order approximation (Taylor expansion) of the above function:

$$\begin{aligned}\mathcal{L}^{(t)} &\approx \sum_{i=1}^n \underbrace{[l(y_i, \hat{y}_i^{(t-1)})]_{\text{constant}}} + g_i f_t(x_i) + \frac{1}{2} h_i f_t^2(x_i) + \Omega(f_t) \\ &\propto \sum_{i=1}^n [g_i f_t(x_i) + \frac{1}{2} h_i f_t^2(x_i)] + \Omega(f_t)\end{aligned}$$

where:

$$g_i = \nabla_{\hat{y}^{(t-1)}} l(y_i, \hat{y}_i^{(t-1)})$$

$$h_i = \nabla_{\hat{y}^{(t-1)}}^2 l(y_i, \hat{y}_i^{(t-1)})$$

XGBoost Algorithm

- Let $I_j := \{i : q(x_i) = j\}$ denote the set of observations that fall into leaf with index j .
- Then the approximation from the previous slide can be written as:

$$\begin{aligned}\tilde{\mathcal{L}}^{(t)} &= \sum_{i=1}^n [g_i f_t(x_i) + \frac{1}{2} h_i f_t^2(x_i)] + \underbrace{\gamma T + \frac{1}{2} \lambda \sum_{j=1}^T w_j^2}_{\Omega(f_t)} \\ &= \sum_{j=1}^T \left[\left(\sum_{i \in I_j} g_i \right) w_j + \frac{1}{2} \left(\sum_{i \in I_j} h_i + \lambda \right) w_j^2 \right] + \gamma T\end{aligned}$$

- For a fixed structure $q(x)$, one can compute optimal weights w_j^* for which the above function achieves a minimum.

XGBoost Algorithm

- Approximation from previous slides:

$$\tilde{\mathcal{L}}^{(t)} = \sum_{j=1}^T \left[\left(\sum_{i \in I_j} g_i \right) w_j + \frac{1}{2} \left(\sum_{i \in I_j} h_i + \lambda \right) w_j^2 \right] + \gamma T$$

- For a fixed structure $q(x)$, optimal weights are:

$$w_j^* = - \frac{\sum_{i \in I_j} g_i}{\sum_{i \in I_j} h_i + \lambda}$$

- The minimum value of $\tilde{\mathcal{L}}^{(t)}$ corresponding to w_j^* , for fixed q , is:

$$\tilde{\mathcal{L}}^{(t)}(q) = -\frac{1}{2} \sum_{j=1}^T \frac{(\sum_{i \in I_j} g_i)^2}{\sum_{i \in I_j} h_i + \lambda} + \gamma T \quad (1)$$

XGBoost Algorithm

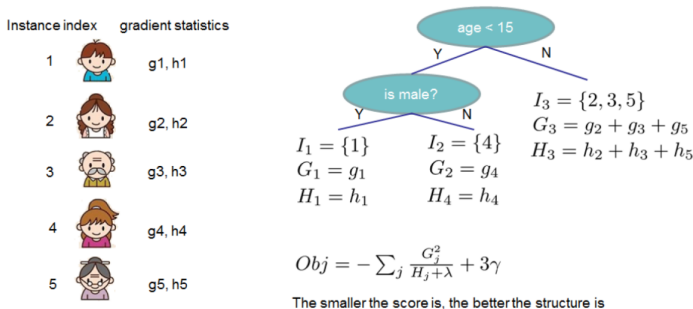
- The minimum value of $\tilde{\mathcal{L}}^{(t)}$ corresponding to w_j^* , for fixed q , is:

$$\tilde{\mathcal{L}}^{(t)}(q) = -\frac{1}{2} \sum_{j=1}^T \frac{(\sum_{i \in I_j} g_i)^2}{\sum_{i \in I_j} h_i + \lambda} + \gamma T \quad (2)$$

- Formula (3) can be used as a function describing the quality of the tree structure q . The smaller the value of (3), the better the tree structure.
- This formula can be treated as an alternative to functions used as splitting criteria (see: $\Delta Q(P)$ in earlier slides) in tree construction.

XGBoost Algorithm

Example calculation of formula (1) describing the quality of the tree structure:



source: <https://arxiv.org/pdf/1603.02754.pdf>

XGBoost Algorithm

Method for finding splits in nodes in the XGBoost algorithm:

- Checking all possible structures q to minimize the function:

$$\tilde{\mathcal{L}}^{(t)}(q) = -\frac{1}{2} \sum_{j=1}^T \frac{(\sum_{i \in I_j} g_i)^2}{\sum_{i \in I_j} h_i + \lambda} + \gamma T \quad (3)$$

is infeasible in practice.

- Therefore, a greedy algorithm is used, which starts from a single node and at each step adds two new nodes.
- Let I_R and I_L denote observations falling into the right and left subtree and $I = I_R \cup I_L$.

As a criterion for splitting observations in node I , we take the difference of values (3):

$$\mathcal{L}_{\text{split}} = \frac{1}{2} \left[\frac{(\sum_{i \in I_L} g_i)^2}{\sum_{i \in I_L} h_i + \lambda} + \frac{(\sum_{i \in I_R} g_i)^2}{\sum_{i \in I_R} h_i + \lambda} - \frac{(\sum_{i \in I} g_i)^2}{\sum_{i \in I} h_i + \lambda} \right] - \gamma$$

XGBoost Algorithm

The XGBoost algorithm includes a number of additional technical elements:

- Weight shrinkage: we multiply the weights computed at each step by a constant η . This constant plays a similar role to the learning rate in optimization algorithms. Shrinkage limits the influence of a given model and "leaves room" for improvement for subsequent trees.
- Random sampling of variables used to build the tree (feature/variable subsampling), a technique used in the RSM method and Random Forests. The goal is to avoid overfitting.
- Algorithm for finding split points based on quantiles of explanatory variables. The goal is to avoid checking all possible split points.
- Parallel computations, operations on sparse matrices.

XGBoost Algorithm

Comparison of XGB, GB and RF:

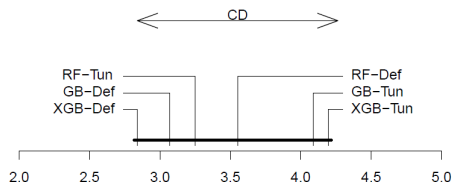
Dataset	D. XGB	T. XGB	D. RF	T. RF	D. GB	T. GB
Australian	86.94%±2.73	87.53%±3.47	87.26%±3.83	86.08%±3.42	86.23%±3.41	86.38%±3.39
Banknote	99.64%±0.59	99.64%±0.49	99.34%±0.60	99.13%±0.71	99.71%±0.48	99.78%±0.33
Breast	96.28%±1.15	95.99%±1.68	96.99%±1.36	96.85%±1.26	96.71%±1.58	96.42%±1.61
Cleveland	81.14%±7.25	83.16%±7.94	82.46%±6.77	82.46%±8.03	83.78%±6.48	82.16%±8.35
Dermatology	96.74%±3.21	97.27%±3.24	97.27%±3.29	97.30%±3.24	96.47%±3.28	96.18%±2.81
Diabetes	75.65%±5.11	76.56%±4.50	76.69%±3.45	76.69%±4.89	76.81%±4.43	76.30%±3.74
Echo	94.46%±6.80	98.75%±3.75	98.75%±3.75	97.32%±5.37	97.32%±5.37	95.89%±6.29
Ecoli	86.87%±5.31	89.05%±4.12	89.07%±5.00	89.11%±4.71	87.25%±6.88	87.81%±4.77
German	79.00%±4.22	77.40%±4.13	76.40%±4.48	75.80%±4.17	76.70%±5.12	77.20%±3.99
Heart	79.26%±5.29	84.07%±5.98	83.33%±5.56	84.44%±5.19	81.85%±7.67	83.70%±5.29
Hepatitis	59.21%±8.28	67.00%±6.56	65.54%±12.35	61.83%±12.68	56.58%±11.90	64.96%±13.08
Ionosphere	92.56%±2.69	92.59%±3.17	93.44%±2.88	93.16%±2.91	93.72%±2.51	92.85%±3.02
Iris	92.67%±6.29	94.00%±4.67	94.67%±4.99	92.67%±6.29	94.67%±4.99	94.67%±4.99
Liver	68.65%±4.69	68.11%±6.12	67.76%±5.03	67.58%±4.07	69.33%±5.26	70.16%±4.42
Magic04	87.47%±0.57	88.63%±0.48	88.19%±0.42	88.18%±0.46	86.85%±0.37	88.83%±0.45
New-thyroid	95.80%±3.26	95.80%±3.26	96.75%±2.94	96.28%±3.48	96.75%±2.94	96.28%±3.48
Parkinsons	92.14%±5.72	90.14%±4.01	90.70%±5.60	90.70%±4.62	91.14%±6.00	91.20%±4.23
Phishing	89.65%±2.44	91.13%±1.30	88.63%±3.38	89.51%±2.42	90.62%±1.76	90.25%±2.19
Segment	98.48%±0.70	98.70%±0.77	98.01%±0.80	98.18%±0.67	97.75%±0.74	98.35%±0.64
Sonar	85.59%±6.44	86.97%±4.84	83.64%±3.87	85.59%±4.83	84.66%±4.61	88.95%±4.32
Soybean	94.80%±3.35	95.22%±2.96	94.06%±2.27	94.65%±2.57	93.63%±2.86	93.58%±3.45
Spambase	95.17%±1.29	95.57%±1.28	95.46%±1.38	95.48%±1.26	94.59%±1.43	96.11%±1.20
Teaching	63.55%±8.00	64.26%±14.15	65.51%±8.95	63.41%±11.13	62.76%±7.38	68.75%±14.65
Tic-tac-toe	96.56%±1.63	100.00%±0.00	95.52%±1.68	95.62%±1.38	90.09%±2.77	100.00%±0.00
Vehicle	78.74%±3.29	77.18%±2.31	74.50%±3.55	74.13%±3.98	78.15%±1.89	77.79%±2.04
Vowel	92.63%±3.53	95.86%±2.14	97.47%±1.37	97.78%±1.68	93.23%±3.32	96.77%±2.01
Waveform	85.72%±1.05	85.72%±1.77	85.42%±1.73	85.62%±1.46	85.32%±0.85	85.86%±1.18
Wine	97.18%±2.83	98.82%±2.37	98.26%±2.66	98.26%±2.66	97.74%±2.78	98.82%±2.37

10

source: Bentejac et. al., 'A Comparative Analysis of XGBoost', <https://arxiv.org/pdf/1911.01914.pdf>

XGBoost Algorithm

Comparison of XGB, GB and RF:



source: Bentejac et. al., 'A Comparative Analysis of XGBoost', <https://arxiv.org/pdf/1911.01914.pdf>

In recent years, modifications of XGBoost have been proposed:

- 1 LightGBM algorithm (Microsoft).
- 2 CatBoost algorithm (Yandex).